

8

Métodos Numéricos

Recuperatorio de parcial

Segundo Cuatrimestre 2020

Alumna: Olivia De Vincenti

Legajo: 60354

Profesor: Pablo Ignacio Fierens

Fecha: 27 de noviembre de 2020

Buenos Aires, Argentina

Consigna

Tema E

- Resolución de EDOs mediante Runge-Kutta 4
- Búsqueda de ceros mediante el método de la bisección
- Ajuste por cuadrados mínimos usando descomposición QR

Funciones

1. Ejercicio 1	1
2. Ejercicio 2	5
3. Ejercicio 3	6

Índice de Figuras

1. Gráfico de la función obtenida	2
2. Gráfico de la derivada de la función obtenida	3
3. Gráfico de la función de ajuste junto con los puntos medidos	8

Índice de Códigos

1. Implementación de Runge Kutta 4	1
2. Definición de la edo	1
3. Estimación del paso	1
4. Resolución de la EDO	2
5. Cálculo del error	3
6. Implementación del método de la bisección	5
7. Llamada a bisec	5
8. Construcción de A	6
9. Implementación de cuadrados mínimos usando descomposición QR	6
10. Resolución al problema	7
11. Función de ajuste	7

1. Ejercicio 1 3-

Se pide resolver la ecuación diferencial dada en el intervalo $t \in [0, 50]$ usando el método Runge-Kutta 4.

Código 1: Implementación de Runge Kutta 4

```

1 def ruku4(edo, t0, tf, h, ci):
2     time = np.arange(t0, tf + h, h)
3     x = np.zeros((len(time), len(ci)))
4     x[0] = ci
5     fvec = np.zeros((4, len(ci)))
6     for i, t in enumerate(time[:-1]):
7         fvec[0] = edo(t, x[i])
8         fvec[1] = edo(t + h / 2, x[i] + h * fvec[0] / 2)
9         fvec[2] = edo(t + h / 2, x[i] + h * fvec[1] / 2)
10        fvec[3] = edo(t + h, x[i] + h * fvec[2])
11        x[i + 1] = x[i] + h * (np.dot(np.array([1, 2, 2, 1]), fvec) / 6)
12    return time, x

```

Utilizando la EDO definida de la forma:

Código 2: Definición de la edo

```

1 def edo(t, x):
2     return np.array([- np.tanh(np.cos(t)*x[0]*x[1]), x[0]])

```

Mal. Pero está bien en el código enviado.

Para estimar el paso, se resolvió el problema usando un paso que converja y luego se lo volvió a resolver usando la mitad del paso anterior. Siguiendo la fórmula para estimar el error:

$$x_{2k}^{(2)} - x_k^{(1)} \approx c \left(1 - \frac{1}{2^n}\right) \Delta t^n \quad (1)$$

Se despeja c y se aplica la siguiente fórmula para estimar el Δt :

$$E_k(\Delta t) \approx c \Delta t^n \quad (2)$$

Despejando Δt^n obtengo un paso apropiado pero como la relación con el error es menor o igual, tomo el 90 % de ese valor para garantizar que el error sea menor a 10^{-6} .

Código 3: Estimación del paso

```

1 # Para estimar el paso, tengo que estimar c, tomo un h cualquiera (que converja)
2 Emax = 1e-6
3 n = 4
4 h1 = 0.5
5 t1, x1 = ruku4(edo, t0, tf, h1, x0)
6 t2, x2 = ruku4(edo, t0, tf, h1/2, x0)
7 carr = np.zeros(len(x1))
8 for i in range(len(carr)):
9     carr[i] = np.abs(x2[2*i][1] - x1[i][1])/((1-1/2**n)*h1**n) # Estimo c para cada valor
10 c = max(carr) # Tomo el máximo, que dará el menor paso
11 print("c estimado:", c)

```

No es necesario un lazo.

```
12 h = 0.9*np.power(Emax/c, 1/n)           # Estimo paso, para garantizar un error
    ↪ menor a Emax,
13 print("h estimado:", h)                 # tomo el 90 %
```

Como resultado de la operación, se obtiene:

$$c = 0.03849082844004457$$

$$\Delta t = 0.06425444767765295$$

Ahora se puede obtener la función:

Código 4: Resolución de la EDO

```
1 x0 = np.array([1.0, 1.0])
2 t0 = 0
3 tf = 50
4 t, x = ruku4(edo, t0, tf, h, x0)       # Resuelvo el problema
```

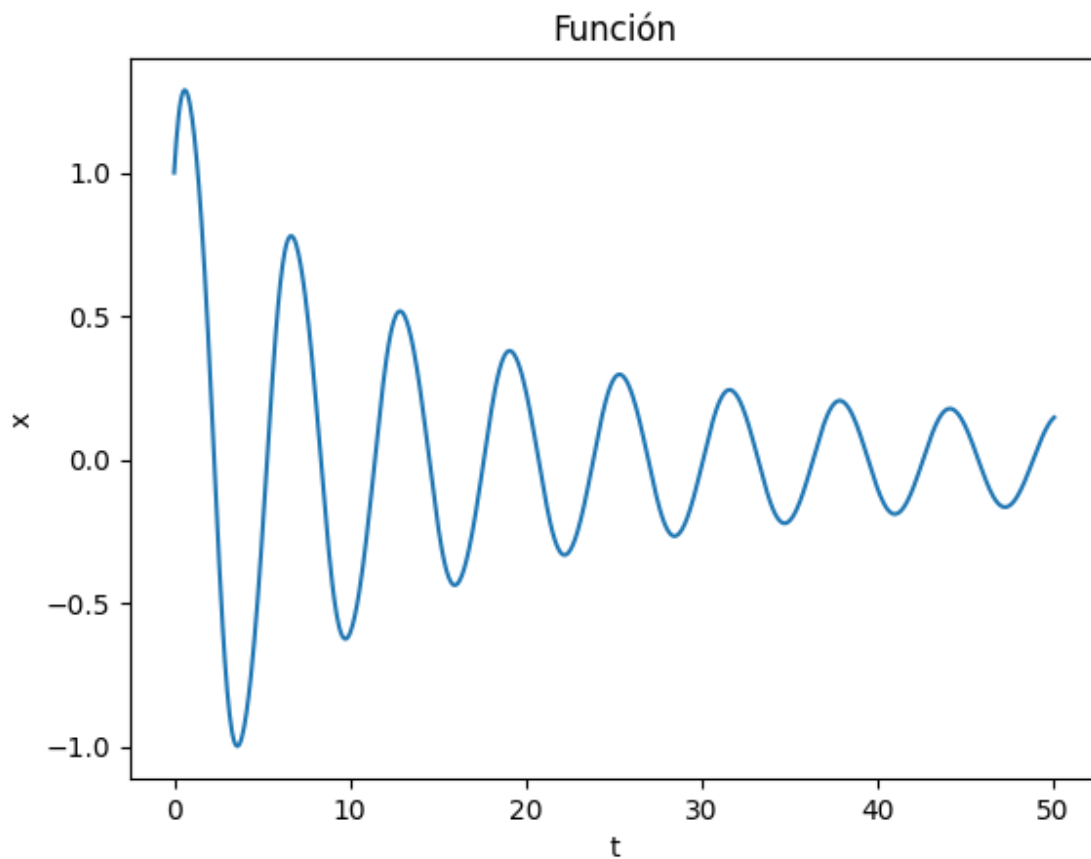


Figura 1: Gráfico de la función obtenida

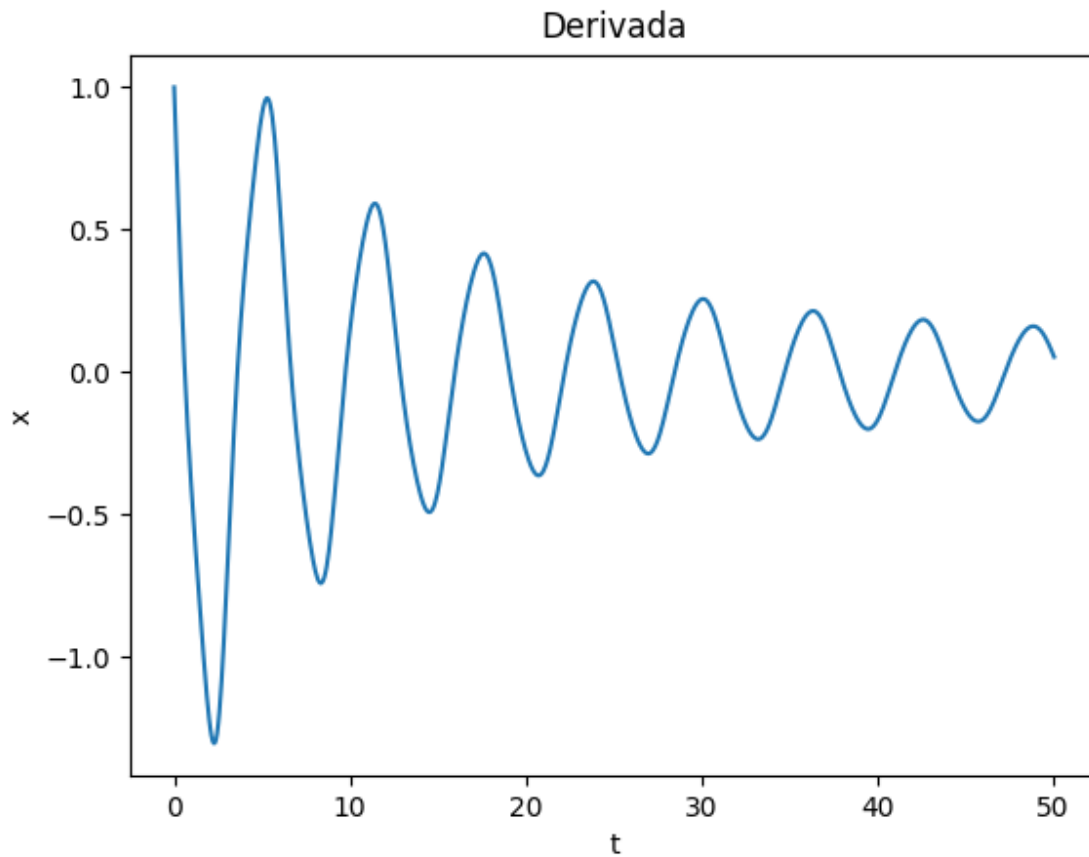


Figura 2: Gráfico de la derivada de la función obtenida

Para calcular el error obtenido, uso la fórmula:

$$E_k \approx \frac{2^n}{\left(1 - \frac{1}{2^n}\right)} \left(x_{2k}^{(2)} - x_k^{(1)}\right) \quad (3)$$

Código 5: Cálculo del error

```

1  # Cálculo del error
2  t_2, x_2 = ruku4(edo, t0, tf, h/2, x0)
3  err_x = np.zeros(len(x))
4  for i in range(len(err_x) - 1):
5      err_x[i] = 2**n * np.abs(x_2[2*i][1] - x[i][1]) / (2**n - 1)  # Estimo el error para cada valor
6  Error_x = max(err_x)  # Busco el máximo
7  print("Error en x:", Error_x)
8
9  err_d = np.zeros(len(x))
10 for i in range(len(err_d) - 1):
11     err_d[i] = 2**n * np.abs(x_2[2*i][0] - x[i][0]) / (2**n - 1)  # Estimo el error para cada valor
12 Error_d = max(err_d)  # Busco el máximo
13 print("Error en x'", Error_d)

```

Esto indica que el error máximo en x es $7.928572602011741e-07$, el cual es menor a 10^{-6} . El error en la derivada es $1.0228702789068696e-06$, es mayo al pedido porque la formula arroja una estimación.

También se podría haber achicado un poco el paso.

2. Ejercicio 2 ²

Se considera el polinomio:

$$f(x) = x^4 + x^3 + x^2 + x - 40 \quad (4)$$

Se pide encontrar el primer cero positivo, entonces sé que mi intervalo debe comenzar en $x = 0$.
Calculo su derivada.

$$f'(x) = 4x^3 + 3x^2 + 2x + 1 \quad (5)$$

Como por consigna $x > 0$, entonces $f'(x)$ será siempre mayor a cero porque ninguno de sus términos está restando. Esto significa que $f(x)$ es monótona creciente en $[0, \infty)$ por lo tanto, según el Teorema de Bolzano, como $f(0) < 0$, f tendrá un único cero en ese intervalo.

Para encontrarlo, voy probando valores hasta que uno me de con signo distinto:

$$f(0) = -40 \quad f(1) = -36 \quad f(2) = -10 \quad f(3) = 80$$

Entonces sé que el primer cero debe estar en el intervalo $(2, 3)$.

Para encontrar el cero de la función, utilizo el método de la bisección:

Código 6: Implementación del método de la bisección

```
1 def bisec(a, b, f, tol=np.finfo(float).eps, max_i=100):
2     newa = a
3     newb = b
4     n = int(np.ceil(np.log2((newb - newa) / tol)) - 1)    # Calcula las iteraciones necesarias para el
        ↪ error pedido
5     print(f"Se precisan {n} iteraciones")
6     if n > max_i:
7         print(f"El número de iteraciones necesario es mayor al máximo, se realizarán {max_i}
            ↪ iteraciones")
8         n = max_i
9     for i in range(n):
10        c = newa + (newb - newa) / 2                        # Calcula c
11        if (f(c) * f(newa)) >= 0:                          # Se fija el signo de f(c)
12            newa = c                                       # Si es igual al de a, a = c
13        else:
14            newb = c                                       # Si es igual al de b, b = c
15        print(f"Raíz en iteración {i + 1}: {(newa + newb) / 2}")
16    return (newa + newb) / 2
```

Es innecesaria la evaluación en $f(\text{newa})$.
La idea de bisección es hacer una sola evaluación de función por iteración.

Para obtener un error menor a 210^{-16} , llamo a la función de la forma:

Código 7: Llamada a bisec

```
1 print(bisec(2, 3, lambda x: x**4 + x**3 + x**2 + x - 40, 2e-16))
```

Como resultado, el primer cero de la función (5) estará en

$$x = 2.1819967597422094$$

3. Ejercicio 3 3

Se busca encontrar los coeficientes que mejor ajusten los puntos del archivo dado a la ecuación:

$$y = a\sqrt{|x|} + b\cos(\sqrt{|x|}) + c \quad (6)$$

Hay que encontrar 3 coeficientes a, b y c , entonces la matriz $\mathbf{A}$ tendrá 3 columnas. Cada una será el resultado función que le corresponde a ese coeficiente aplicado a los distintos valores de x . Por otro lado, $\vec{b}$ será simplemente los valores de y , que representan los resultados de las mediciones.

Código 8: Construcción de $\mathbf{A}$

```
1 A = np.zeros((x.size, 3))          # Armo matriz A
2 A[:, 0] = np.sqrt(abs(x[:]))
3 A[:, 1] = np.cos(np.sqrt(abs(x[:]))
4 A[:, 2] = 1
```

Para resolver el problema se utilizará descomposición QR.

Código 9: Implementación de cuadrados mínimos usando descomposición QR

```
1 #####
2 # leastsq:
3 # Resuelve el problema de cuadrados mínimos usando descomposición QR (argmin ||A.x-b||)
4 # Recibe la matriz A y el vector b. Devuelve el vector x.
5 # -----
6 def leastqr(A, b):
7
8     # Factorización QR
9     QR = desc_qr(A)                # Realizo la descomposición QR
10    Q1 = QR[0]                     # Recupero Q1 de m*n y ortonormal de A
11    R1 = QR[1]                     # Recupero R1 triangular superior de n*n
12
13    # Resuelvo para encontrar x mínima          # R1.x = Q^T.b
14    C = np.dot(np.transpose(Q1), b)           # C = Q1^T.b
15    x = rev_subs(R1, C)                       # R1.x = C
16
17    return x
18 #####
19
20 #####
21 # desc_qr:
22 # Realiza la descomposición QR reducida usando Gram-Schmidt. Devuelve Q1 y R1
23 # -----
24 def desc_qr(A):
25     m = A.shape[0]                # Defino las dimensiones m y n
26     n = A.shape[1]
27
28     Q = np.zeros(shape=(m, n))    # Creo las matrices base
29     R = np.zeros(shape=(n, n))
30
31     for i in range(n):
```

2 veces la misma
cuenta

```

32     p = A[:, i]
33     for k in range(i):           # Calculo las proyecciones (Q[:, k] nunca va a ser nula porque es
    ↪ ortogonal)
34         p = p - (np.dot(A[:, i], Q[:, k])/np.linalg.norm(Q[:, k])) * Q[:, k]   Q[:,k] está normalizado. No tiene sentido
35         Q[:, i] = p/np.linalg.norm(p)                                           # Normalizo la columna dividir por la norma. Además, debería ser
36         R[i, i] = np.linalg.norm(p)                                             # Calculo la diagonal de R la norma al cuadrado.
37         for j in range(i + 1, n):
38             R[i, j] = np.dot(A[:, j], Q[:, i])   # Calculo la esquina superior de R
39                                                     Esta cuenta ya se hizo arriba.
40     return Q, R
41 #####
42
43 #####
44 # rev_subs:
45 # Resuelve un sistema por sustitución hacia atrás (A.x = y). Devuelve x
46 # La matriz A ingresada debe ser triangular superior, y debe ser un vector
47 # -----
48 def rev_subs(A, y):
49     m = A.shape[0]           # Defino m
50     x = np.zeros(shape=(m, 1)) # Crea el vector base
51     for i in range(m):
52         d = 0
53         for j in range(i):
54             d += A[m - i - 1, m - j - 1] * x[m - j - 1]   # Calcula los términos conocidos
55             x[m - i - 1] = (1/A[m - i - 1, m - i - 1])*(y[m - i - 1] - d)   # Calcula una de las componente
56
57     return x
58 #####

```

Aplicando el algoritmo, obtengo los coeficientes.

Código 10: Resolución al problema

```

1 ya = leastqr(A, y) .flatten()           # Busco coeficientes que mejor ajustan a la función
2 print("Coeficientes:", ya)

```

Finalmente, armo la función de ajuste.

Código 11: Función de ajuste

```

1 # Armo función de ajuste
2 def f(x):
3     return ya[0]*np.sqrt(abs(x)) + ya[1]*np.cos(np.sqrt(abs(x))) + ya[2]

```

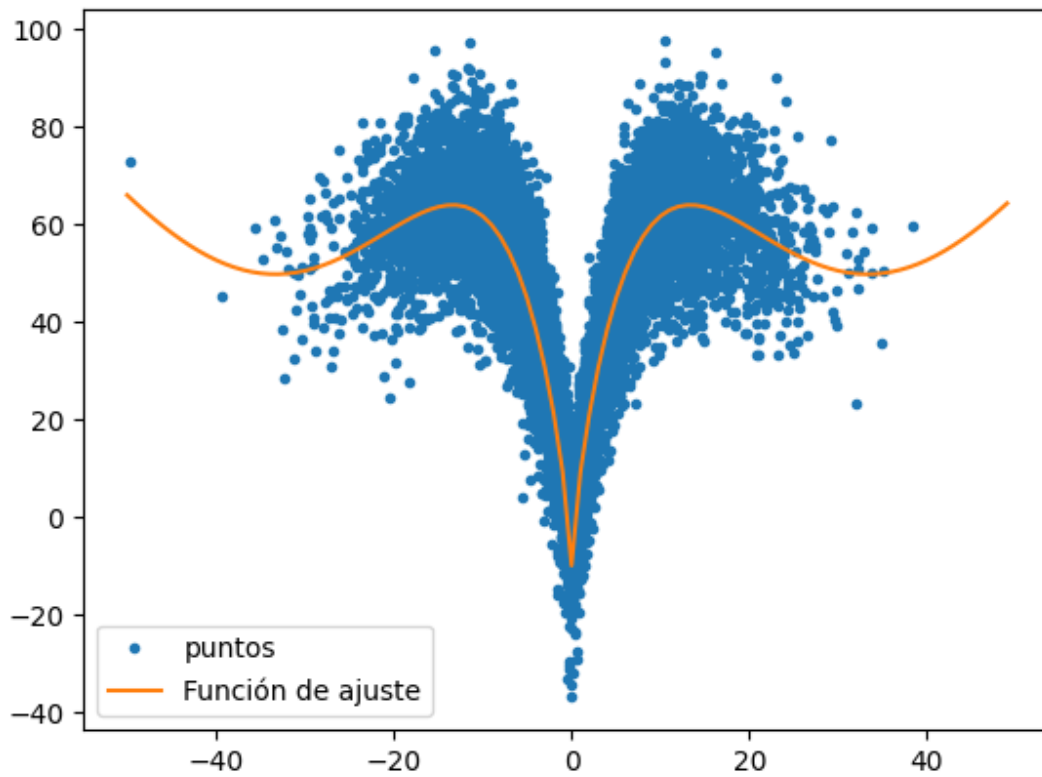


Figura 3: Gráfico de la función de ajuste junto con los puntos medidos

Como se ve en el gráfico, la función ajusta correctamente.

¿Resultados?

Falta el código. -0.5